README

Adaptive Real-Time Traffic Light Simulator Based on Live Vehicle Count

1. Project Description

The Adaptive Real-Time Traffic Light Simulator Based on Live Vehicle Count is a smart traffic-management project that changes traffic signal timings according to the number of vehicles present on different roads. Traditional traffic lights generally use fixed timings, which may not be efficient when traffic conditions change. Our system uses vehicle detection and counting to make the traffic signal more adaptive.

2. Problem Statement

Fixed-time traffic signals can cause unnecessary waiting and congestion because they do not consider the actual traffic density of each lane. A lane with a large number of vehicles may receive the same green-light duration as a lane with very few vehicles. This project aims to provide a more flexible solution by adjusting signal timing based on live vehicle count.

3. Objectives

- Detect vehicles from traffic video in real time.
- Count vehicles present in different lanes.
- Analyze the traffic density of each lane.
- Dynamically adjust traffic signal timing.
- Give higher priority to lanes with heavier traffic.
- Reduce unnecessary waiting and improve traffic flow.
- Demonstrate adaptive software engineering using a real-world application.

4. Main Features

- Real-time vehicle detection

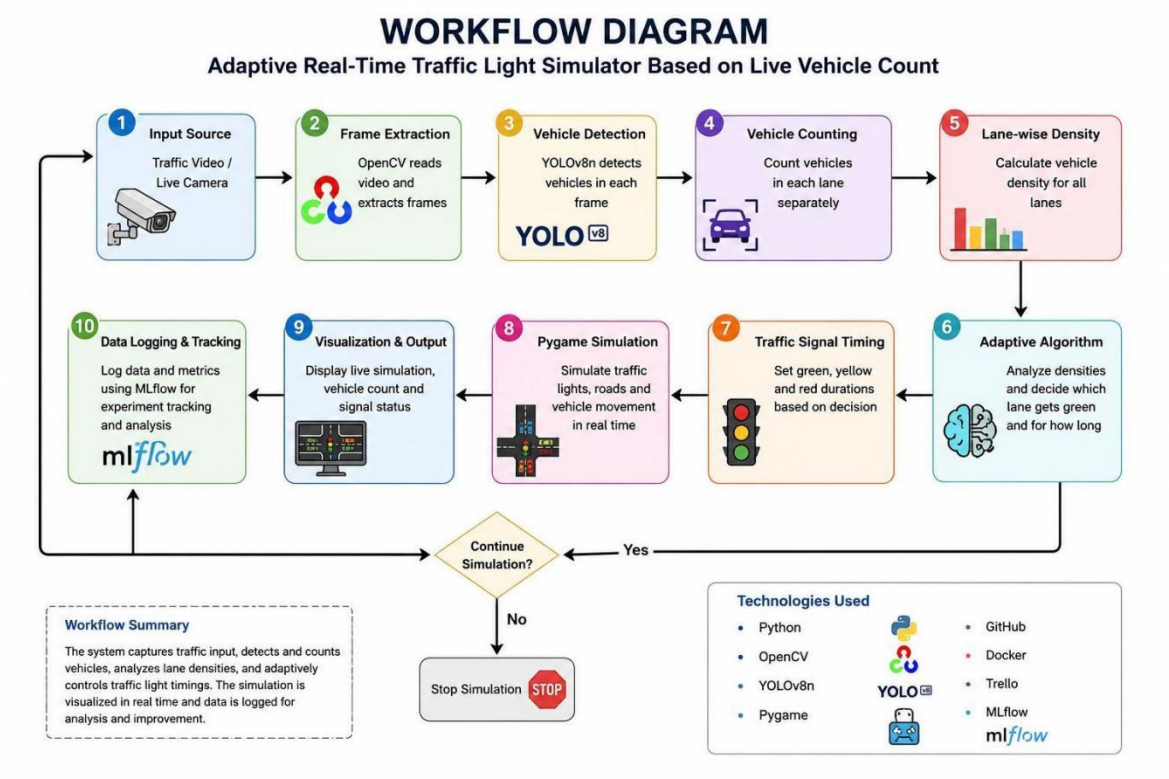
- Vehicle counting
- Lane-wise traffic analysis
- Adaptive traffic-light control
- Dynamic green-light timing
- Real-time traffic simulation
- Visual representation of roads, vehicles, and signals
- Machine-learning experiment tracking

5. Tools and Technologies

Tool	Purpose
Python	Main programming language used for vehicle detection, counting, simulation, and adaptive control logic.
OpenCV	Used for image and video processing and handling traffic-video frames.
YOLOv8n	Used for real-time vehicle detection and identifying vehicles in traffic scenes.
Pygame	Used to create the visual traffic-light and vehicle simulation.
GitHub	Used for source-code storage, version control, and collaboration.
Docker	Used to containerize the application and maintain a consistent execution environment.
Trello	Used for task planning, assignment, progress tracking, and team collaboration.
MLflow	Used to track machine-learning experiments, parameters, metrics, and model versions.

6. System Workflow

Traffic Video/Input → OpenCV Processing → YOLOv8n Vehicle Detection → Vehicle Counting → Lane-wise Traffic Density → Adaptive Algorithm → Traffic Signal Timing → Pygame Simulation



7. How the System Works

1. The system receives traffic video or other vehicle-count input.
2. OpenCV reads and processes the input frames.
3. YOLOv8n detects vehicles in the traffic scene.
4. The detected vehicles are counted for the relevant lanes.
5. The system calculates or compares the traffic density of each lane.
6. The adaptive algorithm identifies which lane requires higher priority.
7. The traffic signal controller assigns an appropriate green-light duration.
8. Pygame displays the traffic-light and vehicle movement simulation.
9. The process repeats as the traffic condition changes.

8. Adaptive Traffic Logic

The adaptive controller compares the number of vehicles in different lanes and gives priority according to traffic density. For example:

Lane 1 → 20 vehicles
Lane 2 → 8 vehicles
Lane 3 → 15 vehicles
Lane 4 → 4 vehicles

In this situation, Lane 1 has the highest traffic density, so the system can give Lane 1 higher priority or a longer green-light duration. When the vehicle count changes, the signal decision can also change.

9. Role of YOLOv8n

YOLOv8n is a lightweight object-detection model used to identify vehicles from traffic images or video. It supports real-time detection and can identify common road vehicles such as cars, buses, trucks, and motorcycles. The detected objects provide the input needed for vehicle counting.

10. Role of OpenCV

OpenCV is used for processing traffic-video frames. It helps read video input, process images, display detection results, and support real-time analysis before the information is passed to the traffic-control logic.

11. Role of Pygame

Pygame is used to build the graphical traffic simulation. It represents roads, lanes, traffic signals, vehicles, and vehicle movement so that the adaptive traffic-control process can be observed visually.

12. Project Management

Trello is used to organize the development process. Project tasks can be divided into categories such as To Do, In Progress, Testing, and Completed. This helps team members assign work and monitor project progress.

13. Version Control

GitHub is used to store and manage the project's source code. It helps the team maintain different versions of the project, collaborate on code, and keep a history of changes.

14. Docker

Docker is used to package the project and its dependencies into a container. This provides a consistent environment and makes the application easier to run across different systems.

15. MLflow

MLflow is used for machine-learning experiment tracking. It can record parameters, metrics, results, and model versions when different detection or machine-learning configurations are tested.

16. Functional Requirements

- The system should accept traffic-video or vehicle-count input.
- The system should detect and count vehicles.
- The system should analyze traffic density for each lane.
- The system should control traffic signals based on the adaptive rules.
- The system should update the simulation according to changing traffic conditions.
- The system should display the traffic state clearly.

17. Non-Functional Requirements

- Usability: The simulator should be simple and easy to understand.
- Performance: Detection and simulation updates should run smoothly.
- Reliability: The adaptive rules should produce consistent signal decisions.
- Maintainability: The code should be organized and easy to modify.
- Scalability: The system should allow future integration with additional lanes, sensors, or models.

18. Testing

The system should be tested with different traffic conditions to verify vehicle detection, vehicle counting, signal changes, and adaptive timing.

Test Case	Condition	Expected Result
Low Traffic	Few vehicles in all lanes	Normal or shorter green duration
Heavy Traffic in One Lane	One lane has significantly more vehicles	Higher priority or longer green duration for that lane
Balanced Traffic	Similar vehicle counts	Fair signal distribution
Changing Traffic	Vehicle counts change during simulation	Signal timing adapts to updated traffic conditions

19. Advantages

- Responds to changing traffic conditions.
- Can reduce unnecessary waiting time.
- Uses real-time vehicle detection.
- Combines computer vision, machine learning, and simulation.
- Provides an interactive demonstration of adaptive traffic control.
- Can be extended toward smart-city traffic management.

20. Limitations

- Detection accuracy depends on video quality and camera position.
- Poor lighting or occlusion may affect vehicle detection.
- The simulator is a model and may not represent every real-world traffic condition.
- Real-world deployment would require suitable sensors, cameras, infrastructure, and safety validation.

21. Future Enhancements

- Integration with live CCTV cameras.
- Emergency-vehicle priority.
- Traffic prediction using machine learning.

- Integration with IoT-based traffic sensors.
- Cloud-based traffic monitoring.
- Accident and road-blockage detection.
- Support for larger and more complex intersections.

22. Project Structure

- main.py – Main Python program.
- vehicle_detection.py – Vehicle detection/counting module, if separated.
- simulation.py – Pygame traffic simulation module, if separated.
- models/ – YOLOv8n model files or model configuration, if required.
- requirements.txt – Python dependencies.
- Dockerfile – Docker configuration.
- README.md – Project documentation.
- Trello – Project task management.
- MLflow – Machine-learning experiment tracking.

23. Installation and Setup

10. Install Python and the required project dependencies.
11. Clone or download the project from GitHub.
12. Install or configure the YOLOv8n model.
13. Provide a traffic video, camera input, or configured vehicle-count data.
14. Run the vehicle-detection and counting module.
15. Start the Pygame traffic simulation.
16. Use Docker if the project is configured for containerized execution.
17. Use MLflow when tracking machine-learning experiments.

24. Sample Use Case

Suppose an intersection has four lanes. If Lane 1 contains 25 vehicles, Lane 2 contains 7 vehicles, Lane 3 contains 12 vehicles, and Lane 4 contains 4 vehicles, the adaptive controller identifies Lane 1 as the most congested lane. It can provide Lane 1 with a higher green-light duration. After the vehicles move and the counts change, the controller re-evaluates the traffic and makes the next decision.

25. Conclusion

The Adaptive Real-Time Traffic Light Simulator Based on Live Vehicle Count demonstrates how adaptive software engineering can be applied to a real-world traffic problem. YOLOv8n and OpenCV are used for vehicle detection and analysis, Python implements the core logic, and Pygame provides the traffic simulation. GitHub, Docker, Trello, and MLflow support version control, deployment, project management, and machine-learning experiment tracking. The project can be further developed into a more advanced intelligent traffic-management system.

26. Team Members

S.No	Name	Roll Number
1.	P.Likitha	2420030028
2.	G.Karthika	2420030076
3.	K.L Manasa	2420030671
4.	J.Vardhini	2420030677